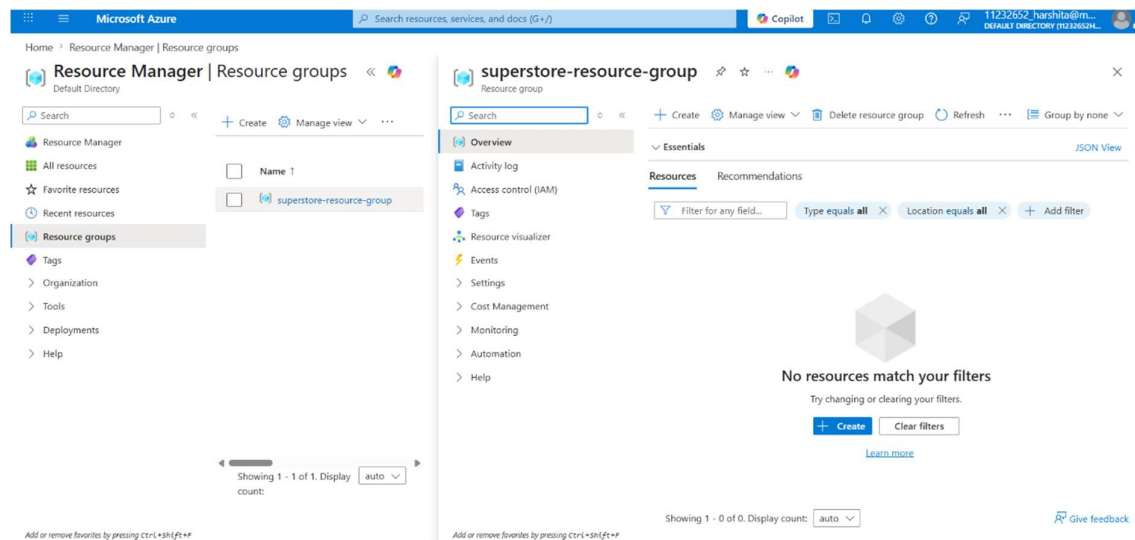


WEEK 4 ASSIGNMENT

Objective: Understand Azure cloud concepts and build an end-to-end data pipeline using Storage Account and Azure Data Factory.

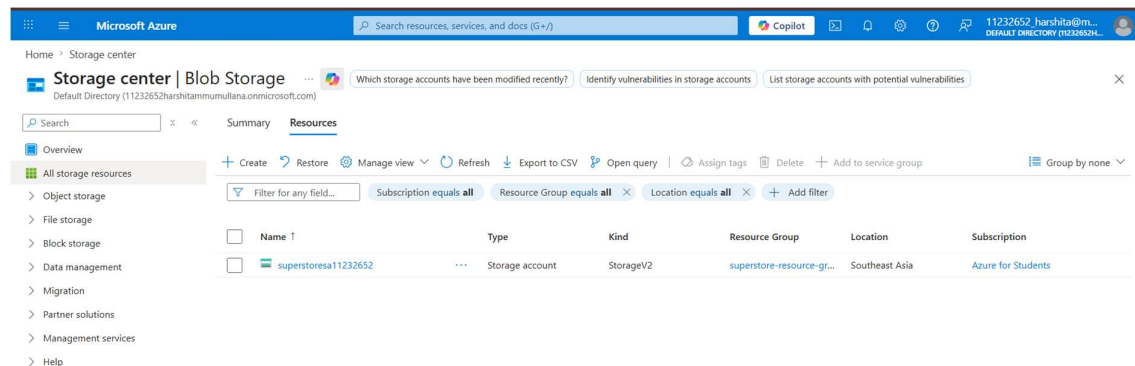
Step 1: Explore Azure Portal and Create a Resource Group

I began by exploring the Azure Portal and familiarizing myself with its interface. I then navigated to Resource Groups and created a new resource group named `superstore-resource-group` under my Default Directory. This resource group served as the logical container for all the Azure resources I created throughout this assignment.



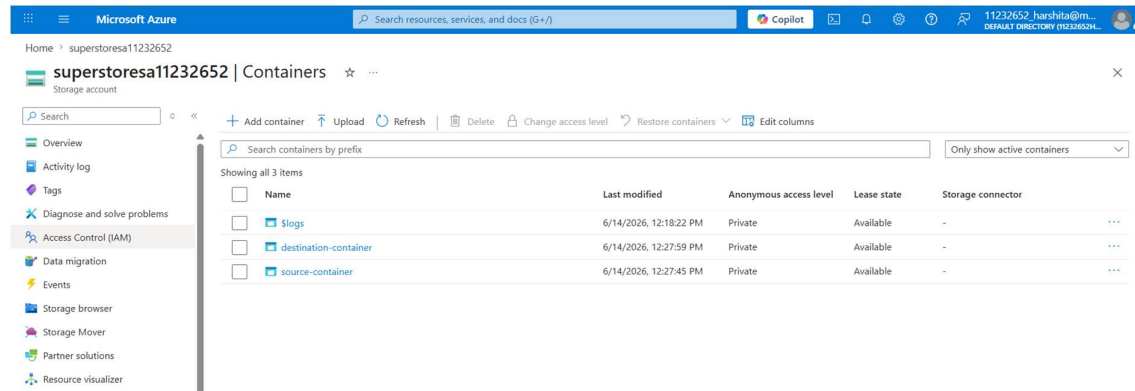
Step 2: Create a Storage Account

I created a Storage Account named `superstoresa11232652` inside `superstore-resource-group`, deployed in the Southeast Asia region.



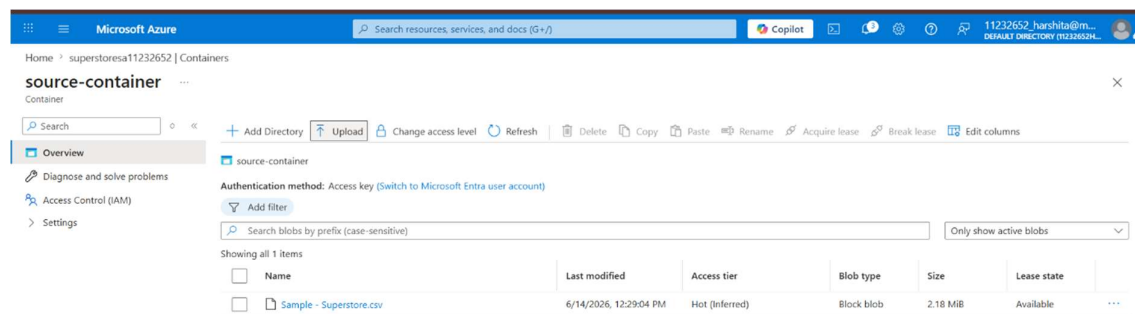
Step 3: Create Blob Containers

After the storage account was ready, I created two Blob Containers inside it — source-container where I uploaded the Superstore dataset CSV downloaded from Kaggle, and destination-container which I kept empty as the target location for the pipeline output. The resource group at this point contained the storage account listed under Southeast Asia.



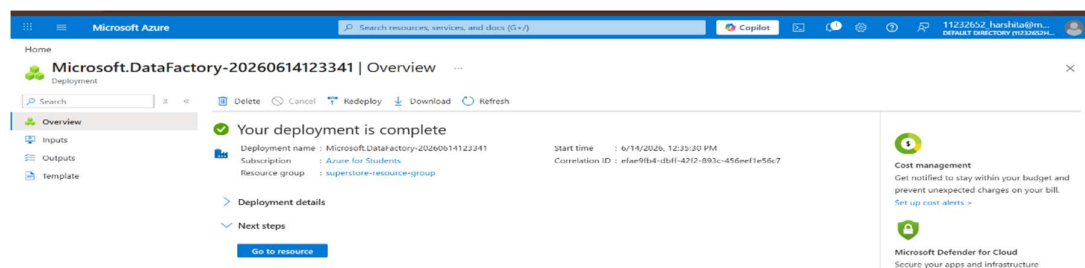
Step 4: Upload CSV File

Then I uploaded the dataset that I had downloaded from Kaggle .



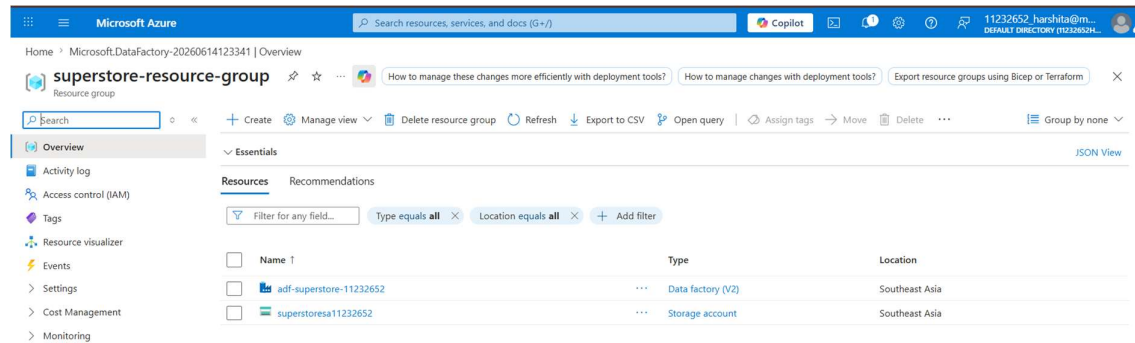
Step 5: Create Azure Data Factory

I created an Azure Data Factory (V2) instance named adf-superstore-11232652 in the same resource group. Before provisioning, I reviewed the configuration on the Review + Create tab — subscription set to Azure for Students, resource group set to superstore-resource-group, region set to Southeast Asia, and networking set to public endpoint.



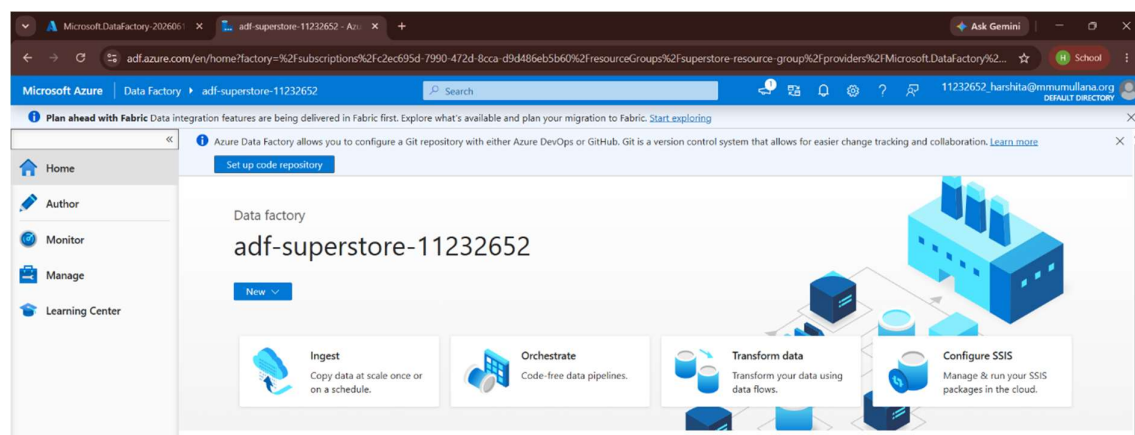
Step 6: Verify Resource Group with All Resources

After the ADF deployment completed, I went back to my resource group to verify that both resources were present — adf-superstore-11232652 (Data Factory V2) and superstorea11232652 (Storage Account), both deployed in Southeast Asia. This confirmed my environment was fully set up.



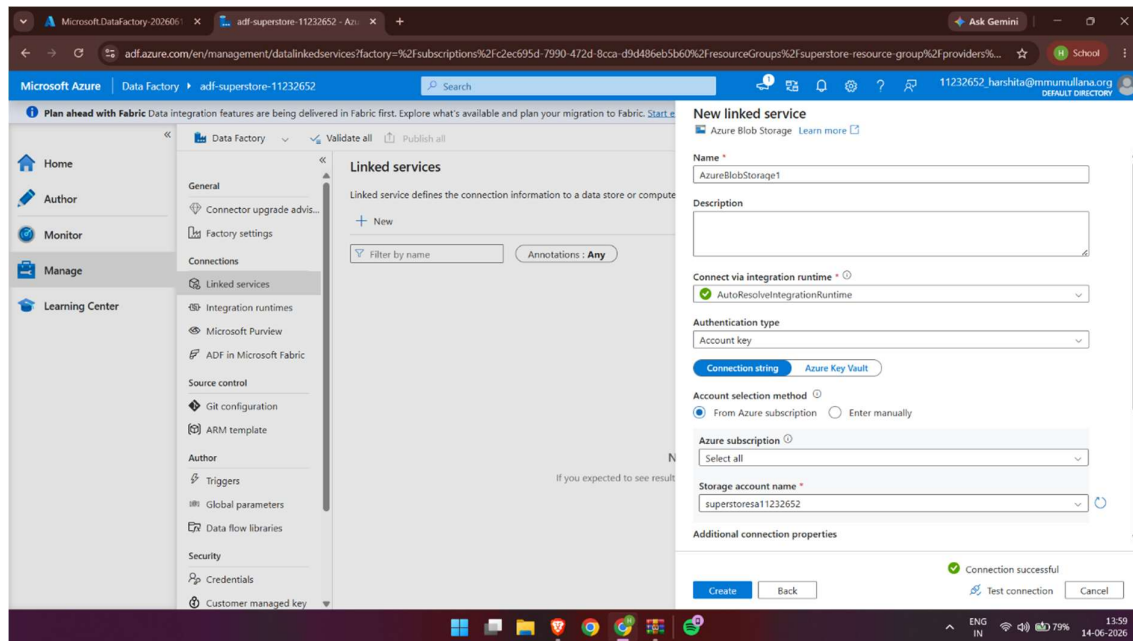
Step 7: Explore ADF UI

I launched the Azure Data Factory Studio and explored its interface. The home screen showed the four capability areas — Ingest, Orchestrate, Transform, and Configure SSIS. I explored the left navigation which has Author, Monitor, Manage, and Learning Center sections, giving me a complete view of what ADF offers for building and managing data pipelines.



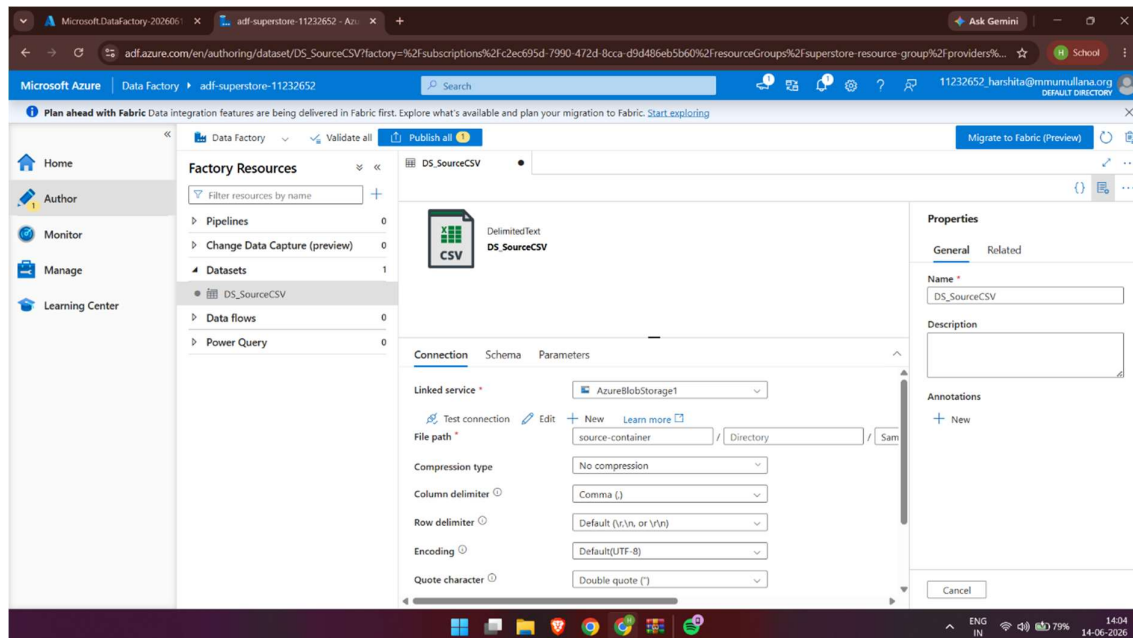
Step 8: Create Linked Service

I went to the Manage tab and created a new Linked Service named AzureBlobStorage1 of type Azure Blob Storage. I connected it to my storage account superstorea11232652 using Account Key authentication via AutoResolveIntegrationRuntime. The connection test at the bottom returned Connection successful before I clicked Create.

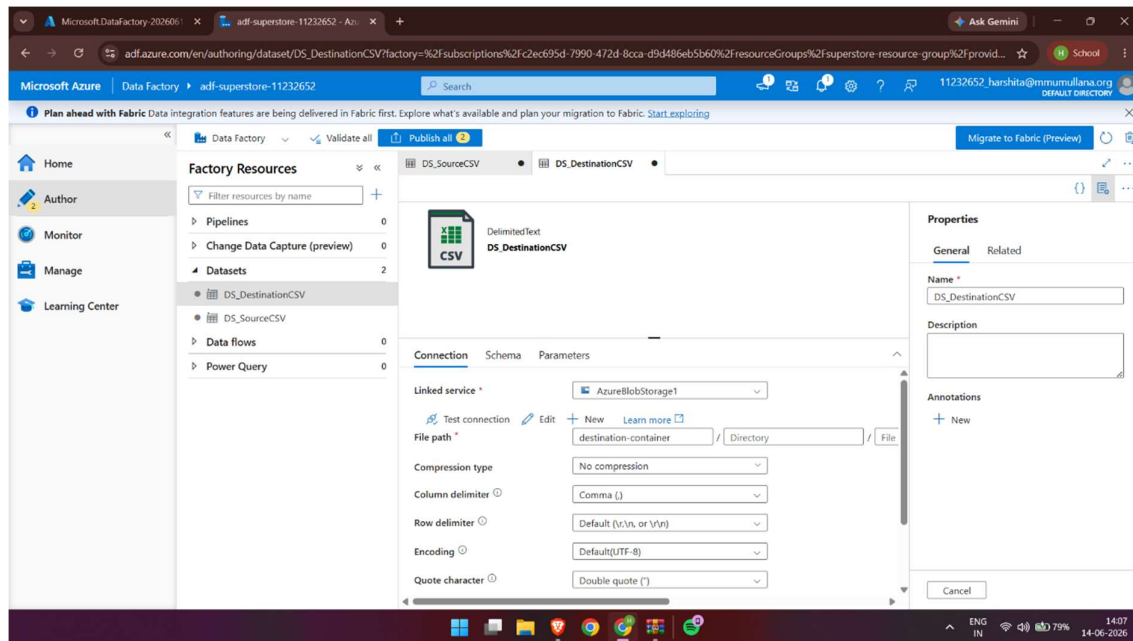


Step 9: Create Source and Destination Datasets

I created my first dataset named DS_SourceCSV of type DelimitedText under the Author tab. I configured it to use AzureBlobStorage1 as the linked service and pointed the file path to source-container where the Superstore CSV was uploaded. I set comma as the column delimiter, UTF-8 encoding, and double-quote as the quote character.



I then created a second dataset named DS_DestinationCSV of type DelimitedText, also using AzureBlobStorage1 as the linked service. I pointed the file path to destination-container as the output location. The same format settings were applied — comma delimiter, UTF-8 encoding, and double-quote character — to ensure consistency between source and destination.



Step 10: Get Metadata Activity — Execution Output

After the Get Metadata activity executed successfully, it returned the following output confirming the source file exists and retrieving its properties:

```
{
  "exists": true,
  "itemName": "Sample - Superstore.csv",
  "itemType": "File",
  "size": 2287806,
  "lastModified": "2026-06-14T06:59:04Z",
  "effectiveIntegrationRuntime": "AutoResolveIntegrationRuntime (Southeast Asia)",
  "executionDuration": 0,
  "durationInQueue": {
    "integrationRuntimeQueue": 0
  },
  "billingReference": {
    "activityType": "PipelineActivity",
    "billableDuration": [
      {
        "meterType": "AzureIR",
        "duration": 0.016666666666666666,

```

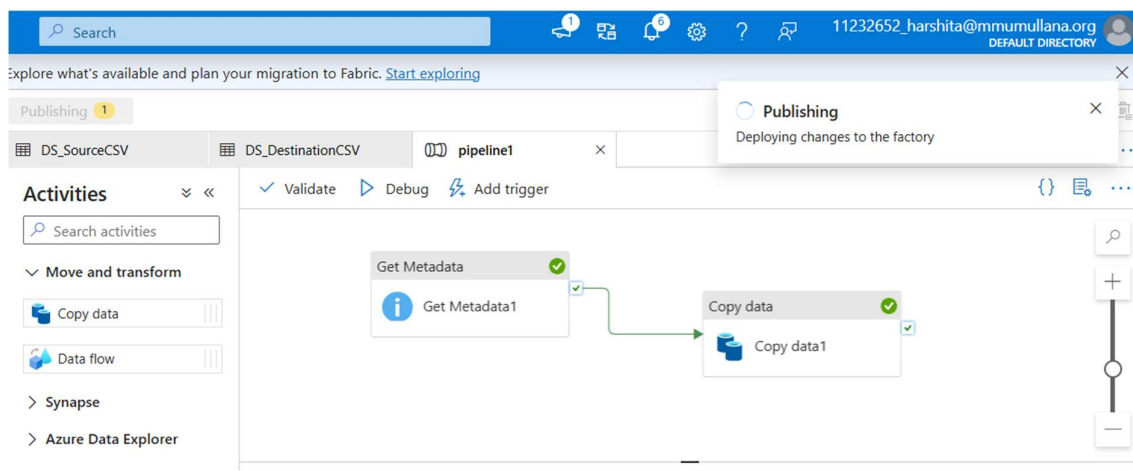
```

        "unit": "Hours"
    }
}
]
}
}

```

Step 11: Build Pipeline Using Copy Data Activity

I added a Copy data activity to the pipeline canvas and connected it to the success output of Get Metadata1. I configured the source as DS_SourceCSV and the destination as DS_DestinationCSV. The pipeline canvas showed both activities linked in sequence and I published all changes to the factory.



Step 12: Execute Pipeline and Monitor Runs

I executed the pipeline using the Debug trigger and switched to the Monitor tab to track execution. The Pipeline runs list showed 14 successful runs recorded throughout the session, all triggered manually, with durations ranging from 14 to 36 seconds. This confirmed the pipeline was stable and repeatable.

The screenshot shows the 'Monitor' tab in Azure Data Factory, displaying a list of pipeline runs for 'pipeline1'. The table includes columns for Pipeline name, Run start, Run end, Duration, Status, Triggered by, and Run ID. All 14 runs shown are 'Succeeded' and were triggered manually. The runs occurred between 6/14/2026 3:28:41 and 6/14/2026 3:36:28.

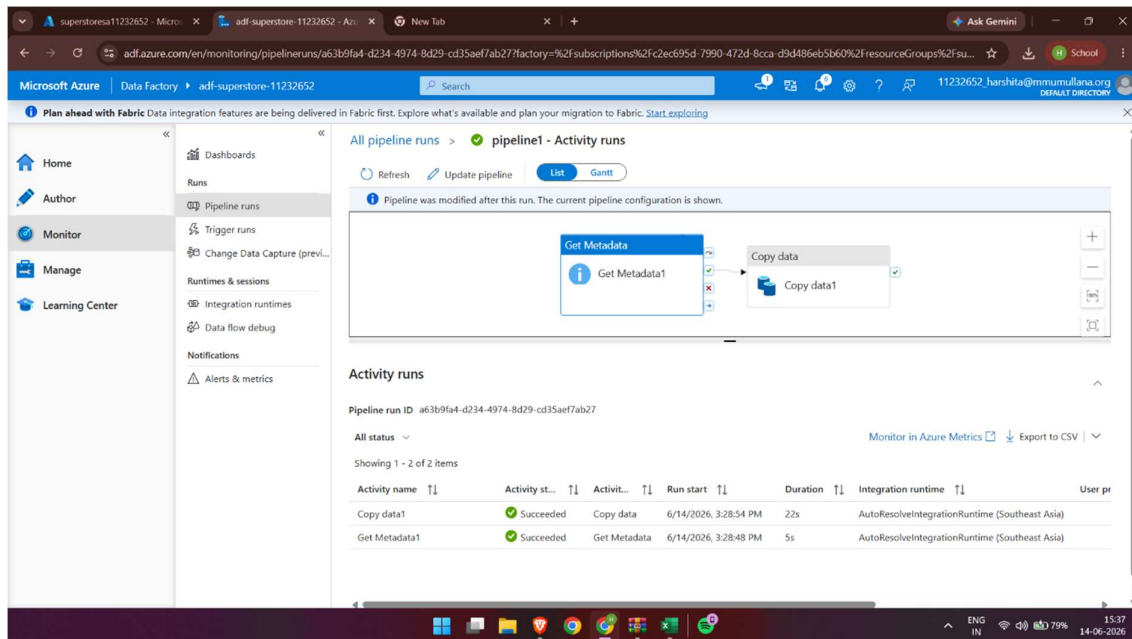
Pipeline name	Run start	Run end	Duration	Status	Triggered by	Run ID
pipeline1	6/14/2026, 3:28:41	6/14/2026, 3:29:16	36s	Succeeded	Manual trigger	a63b9fa4-d234-4974...
pipeline1	6/14/2026, 3:27:46	6/14/2026, 3:28:11	25s	Succeeded	Manual trigger	d23ef267-364d-4a3...
pipeline1	6/14/2026, 3:25:46	6/14/2026, 3:26:00	14s	Succeeded	Manual trigger	48f67765-17f5-45d2...
pipeline1	6/14/2026, 3:24:41	6/14/2026, 3:24:59	19s	Succeeded	Manual trigger	f6f93b4e-cb30-4535...
pipeline1	6/14/2026, 3:19:32	6/14/2026, 3:19:46	15s	Succeeded	Manual trigger	6c3f9b13-eabf-4266...
pipeline1	6/14/2026, 3:18:07	6/14/2026, 3:18:20	14s	Succeeded	Manual trigger	b47b5902-4d6a-49b...
pipeline1	6/14/2026, 3:17:33	6/14/2026, 3:17:54	22s	Succeeded	Manual trigger	d7f1b74d-7ef1-4ce9...
pipeline1	6/14/2026, 3:13:38	6/14/2026, 3:13:57	20s	Succeeded	Manual trigger	b84a10ff-69d1-469f...
pipeline1	6/14/2026, 3:10:41	6/14/2026, 3:11:01	20s	Succeeded	Manual trigger	1c82cec8-f503-457f...
pipeline1	6/14/2026, 3:04:43	6/14/2026, 3:05:11	28s	Succeeded	Manual trigger	9b81deed-5bbb-425...
pipeline1	6/14/2026, 3:01:58	6/14/2026, 3:02:27	29s	Succeeded	Manual trigger	1d218d6-22de-462...
pipeline1	6/14/2026, 2:49:04	6/14/2026, 2:49:32	29s	Succeeded	Manual trigger	56142f87-9303-426d...
pipeline1	6/14/2026, 2:35:54	6/14/2026, 2:36:28	34s	Failed	Manual trigger	df649aba-1rdb-4ffa...

Step 13: View Activity Run Details

I drilled into the latest pipeline run to inspect individual activity results. The activity run detail view showed:

- Get Metadata1 — Succeeded in 5 seconds
- Copy data1 — Succeeded in 22 seconds

Both ran on AutoResolveIntegrationRuntime in Southeast Asia. Pipeline Run ID: a63b9fa4-d234-4974-8d29-cd35aef7ab27.

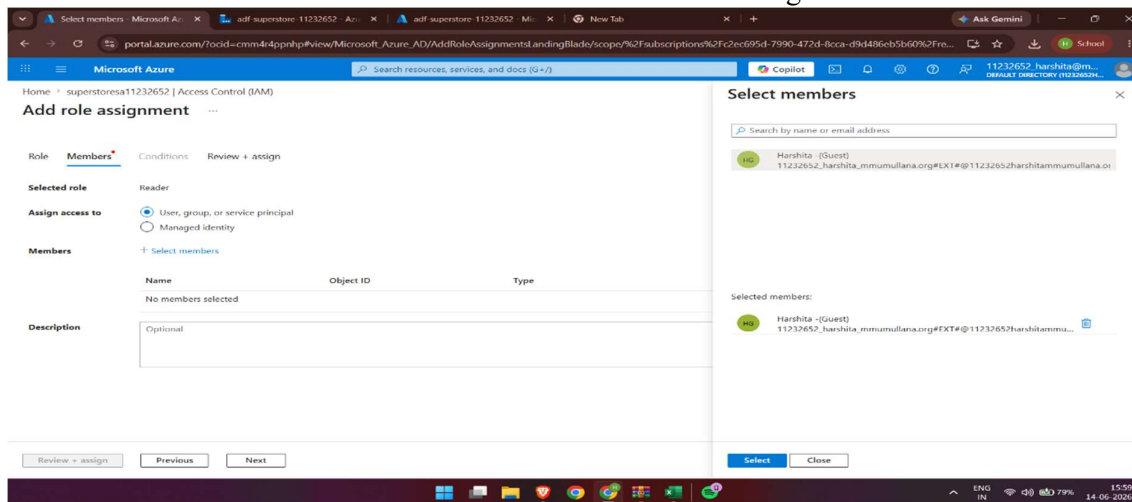


The screenshot displays the Microsoft Azure Data Factory portal. The left sidebar shows navigation options: Home, Author, Monitor, and Manage. The 'Monitor' section is active, showing 'All pipeline runs' for a specific pipeline. The pipeline configuration is shown as a graph with two activities: 'Get Metadata1' and 'Copy data1'. Below this, the 'Activity runs' section shows a table of runs for the pipeline run ID 'a63b9fa4-d234-4974-8d29-cd35aef7ab27'.

Activity name	Activity status	Activity name	Run start	Duration	Integration runtime	User principal
Copy data1	Succeeded	Copy data	6/14/2026, 3:28:54 PM	22s	AutoResolveIntegrationRuntime (Southeast Asia)	
Get Metadata1	Succeeded	Get Metadata	6/14/2026, 3:28:48 PM	5s	AutoResolveIntegrationRuntime (Southeast Asia)	

Step 14: Assign IAM Reader Role

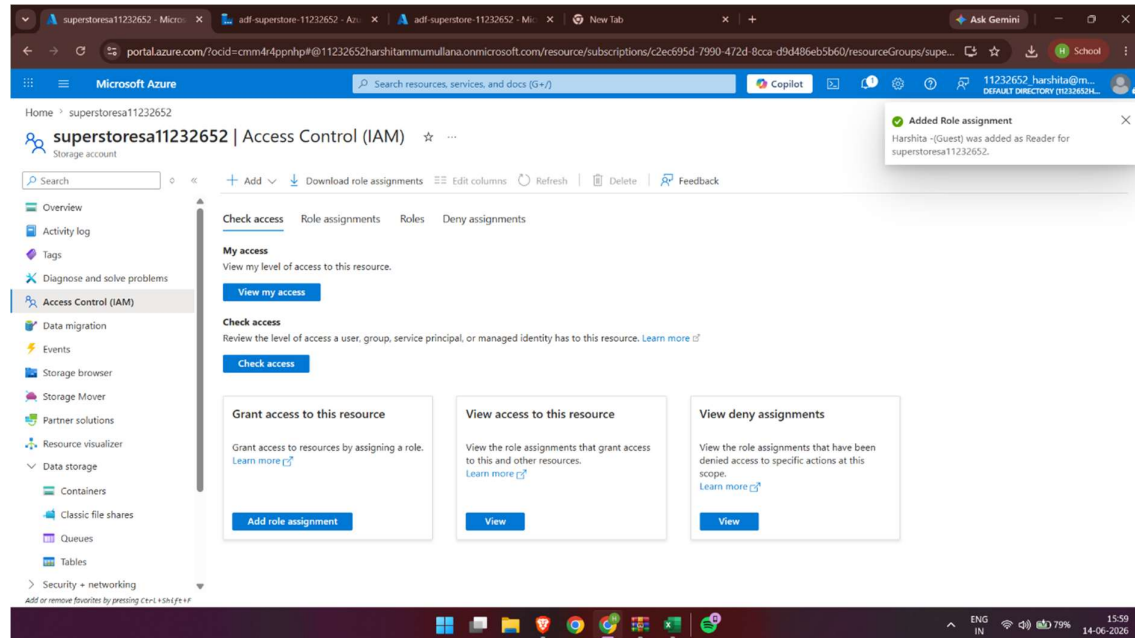
I navigated to the Access Control (IAM) section of my Storage Account and initiated a new role assignment. I selected the Reader role under Job function roles and chose Harshita (Guest) as the member to assign it to.



The screenshot shows the Microsoft Azure portal's 'Add role assignment' page. The 'Role' is set to 'Reader'. The 'Assign access to' section shows 'User, group, or service principal' selected. The 'Members' section shows 'No members selected'. The 'Description' field is optional. On the right, the 'Select members' dialog is open, showing a search bar and a list of members. 'Harshita (Guest)' is selected as the member to assign the role to.

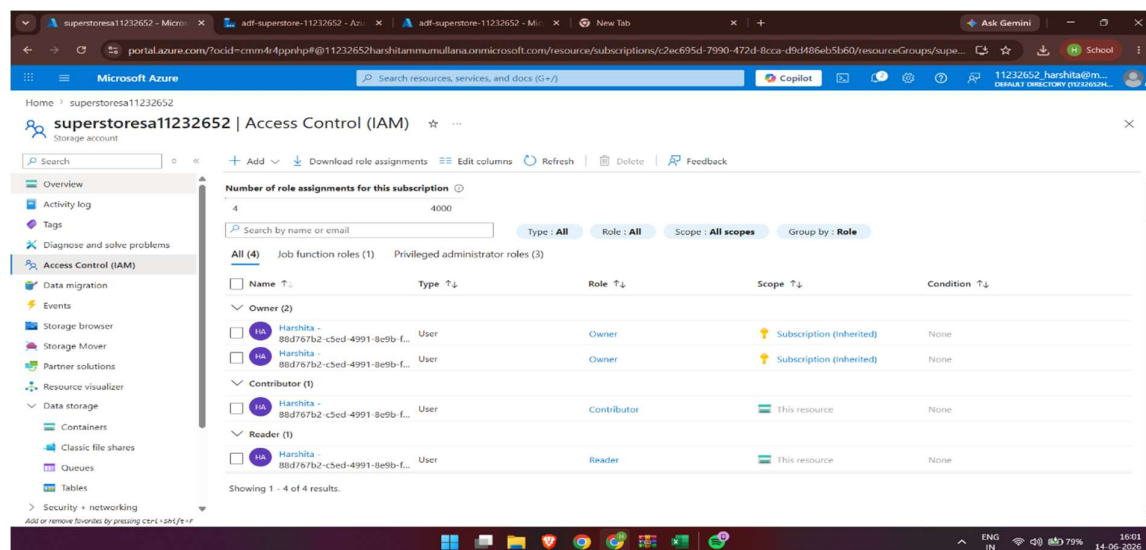
Step 15: Confirm Role Assignment

After submitting, I received a confirmation notification: "Added Role assignment — Harshita (Guest) was added as Reader for superstorea11232652." This confirmed the Reader role was successfully applied.



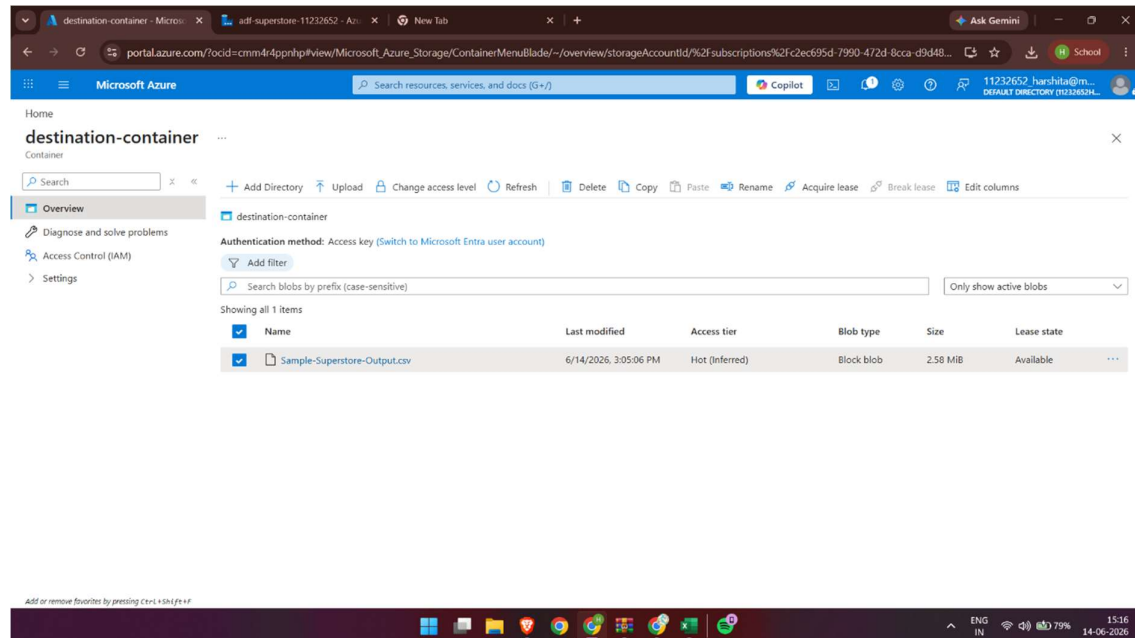
Step 16: View All IAM Role Assignments

I reviewed the complete Role Assignments list which showed 4 total assignments — Owner $\times 2$ (inherited from Subscription), Contributor $\times 1$ (resource scope), and Reader $\times 1$ (resource scope) — confirming proper RBAC access was configured between ADF and the Storage Account.



Step 17: Verify Output in Destination Container

I navigated to destination-container in my Storage Account to verify the pipeline output. The file Sample-Superstore-Output.csv was present with a size of 2.58 MiB, stored as a Block Blob in the Hot access tier. This confirmed the full end-to-end pipeline — source Blob → Get Metadata validation → ADF Copy → destination Blob — completed successfully.



Summary

In this assignment, I built an end-to-end data pipeline on Microsoft Azure using Azure Blob Storage and Azure Data Factory. I started by creating a Resource Group named superstore-resource-group to organize all my resources, then provisioned a Storage Account with two Blob Containers — source-container where I uploaded the Superstore dataset from Kaggle, and destination-container as the pipeline output location.

I then created an Azure Data Factory instance named adf-superstore-11232652 and explored its UI — Author, Monitor, and Manage sections. Inside ADF, I created a Linked Service AzureBlobStorage1 to connect to my storage account, and two datasets DS_SourceCSV and DS_DestinationCSV pointing to the respective containers.

I built pipeline1 using two activities — **Get Metadata** to validate the source file before copying, and **Copy Data** to transfer the Superstore CSV from source to destination. I executed the pipeline using the Debug trigger and monitored it in the Monitor tab, recording 14 successful runs with the final run completing in 36 seconds.

Finally, I configured IAM roles on the Storage Account — assigning Reader and Contributor roles — to ensure proper access between ADF and Blob Storage. The output file Sample-Superstore-Output.csv (2.58 MiB) was successfully written to the destination container, completing the full **Blob → ADF → Blob** pipeline with metadata validation.